having access control in the first place. Ruby provides the related method public_send, which will only send to methods that are public. For most usages, public_send is preferred.

Another way of invoking methods dynamically uses Method objects. A Method object is like a Proc object: it represents a chunk of code and a context in which it executes. In this case, the code is the body of the method, and the context is the object that created the method. Once we have our Method object, we can execute it sometime later by sending it the message call:

```
trane = "John Coltrane".method(:length)
miles = "Miles Davis".method("sub")

trane.call              # => 13
miles.call(/iles/, '.') # => "M. Davis"
```

You can pass the Method object around as you would any other object, and when you invoke call, the method is run just as if you had invoked it on the original object. It's like having a C-style function pointer but in a fully object-oriented style. Or, if you are familiar with JavaScript or Python, in those languages, mentioning a method without arguments returns the method itself, whereas in Ruby, mentioning the method without arguments calls the method with no arguments, and if you want the method itself, you need to call method.

You can use Method objects where you could use proc objects. For example, they work with iterators:

```
def double(a)
  2 * a
end

method_object = method(:double)
[ 1, 3, 5, 7 ].map(&method_object) # => [2, 6, 10, 14]
```

Method objects are bound to one particular object and that object acts as the receiver when the method object is invoked. You can create *unbound* methods (of class UnboundMethod) by calling instance_method on a class with the name of the method. Then subsequently, you can bind the method to one or more objects with bind. The binding creates a new Method object, that is bound to the argument of bind. As with aliases, unbound methods are references to the definition of the method at the time they are created and do not reflect later changes:

```
unbound_length = String.instance_method(:length)
  class String
    def length
      99
    end
   end

str = "cat"
str.length        # => 99

bound_length = unbound_length.bind(str)
bound_length.call # => 3
```

The pattern of binding a method and immediately calling it is common enough that Ruby has a shortcut, bind_call. We can replace the last two lines of the above snippet with the single line unbound_length.bind_call(str). The first argument to bind_call is the object to bind to, and any subsequent arguments are passed to the call.